

即時競標與限時搶購系統

Real-time Bidding & Flash Sale System

分散式系統與雲端應用開發實務 期末專題報告

目錄

- 1. 專案概述
- 2. 系統架構
- 3. 核心功能與實作
- 4. 技術棧
- 5. 關鍵技術挑戰與解決方案
- 6. 開發歷程與試誤改進
- 7. 壓力測試與效能驗證
- 8. 部署架構
- 9. 專案結構
- 10. 總結與心得

1. 專案概述

1.1 專案背景

本系統模擬「雙十一」、「黑色星期五」等大型電商促銷活動情境，設計一個能夠承受瞬間巨大流量的雲端後端系統，解決「驚群效應」(Thundering Herd Problem) 帶來的技術挑戰。

1.2 專案目標

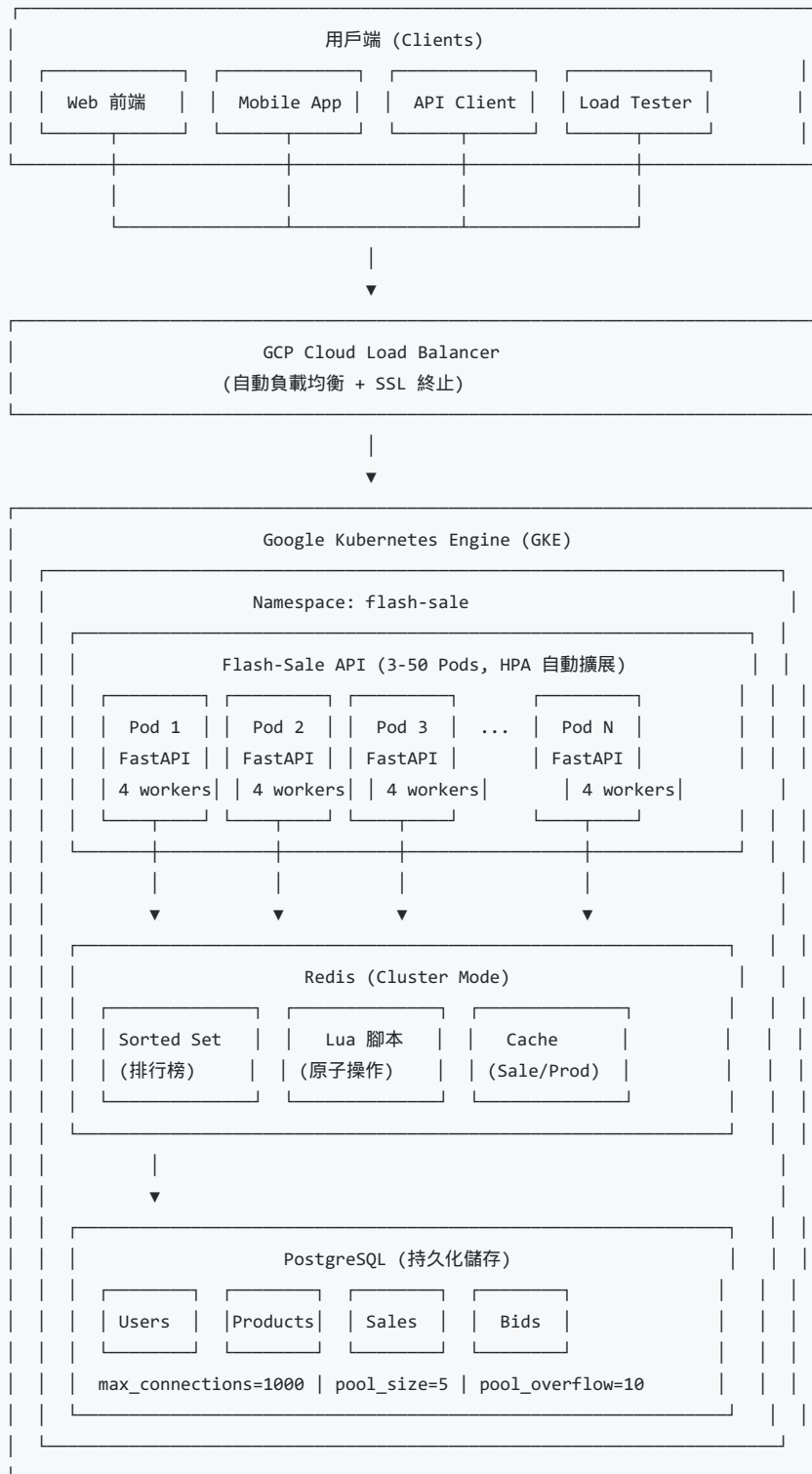
需求項目	目標值	達成狀態
並發用戶數	≥ 1000 users	達成
防超賣保證	100% 不超賣	達成
即時排行榜	毫秒級更新	達成
自動擴展	動態 Pod 擴縮	達成
雲端部署	GCP GKE	達成

1.3 線上環境

服務	URL	說明
API Server	http://34.81.236.6	RESTful API 端點
Frontend	http://35.189.162.209	Web 操作介面

2. 系統架構

2.1 整體架構圖



2.2 資料流程圖



3. 核心功能與實作

3.1 評分公式 (Scoring Formula)

本系統的得標機制不僅考慮出價金額，更綜合考量反應速度與會員貢獻度：

$$\text{Score} = \alpha \times P + \frac{\beta}{(T + 1)} + \gamma \times W$$

參數	名稱	說明	範例值
P	Price	用戶出價金額	1000
T	Time	響應時間 (毫秒)	500

參數	名稱	說明	權重 (1.0-5.0)	範例值
α	Alpha	價格權重係數	1.0	5.0
β	Beta	時間權重係數	10000	
γ	Gamma	會員權重係數	50	
K	Inventory	庫存限制/得標人數	100	

評分計算實作：

```
# app/services/score_calculator.py
def calculate_score(
    price: float,
    response_time_ms: int,
    user_weight: float,
    alpha: float,
    beta: float,
    gamma: float
) -> float:
    """
    計算搶購積分

    Score =  $\alpha \times P + \beta / (T + 1) + \gamma \times W$ 
    """
    price_component = alpha * price
    time_component = beta / (response_time_ms + 1)
    weight_component = gamma * user_weight

    return price_component + time_component + weight_component
```

3.2 防超賣機制 (Anti-Overselling)

使用 **Redis Lua 腳本** 實現原子性操作，確保 100% 不超賣：

```
-- Redis Lua Script: 原子性更新排行榜
local leaderboard_key = KEYS[1]
local user_id = ARGV[1]
local score = tonumber(ARGV[2])
local max_winners = tonumber(ARGV[3])

-- 步驟 1: 原子性更新分數 (如果已存在會覆蓋)
redis.call('ZADD', leaderboard_key, score, user_id)

-- 步驟 2: 原子性取得排名 (0-indexed, 從高分到低分)
local rank = redis.call('ZREVRANK', leaderboard_key, user_id)

-- 步驟 3: 原子性判斷是否得標 (排名 < max_winners)
local is_winner = 0
if rank ~= nil and rank < max_winners then
    is_winner = 1
end

-- 返回結果: [是否得標, 排名]
return {is_winner, rank}
```

防止超賣機制選擇原因：

方式	問題	結果
分開執行 ZADD + ZREVRANK	兩個操作之間可能有其他請求插入	可能出現超賣的現象
Redis Transaction (MULTI/EXEC)	無法在 Transaction 中取得中間結果	無法判斷先後排名

方式	問題	結果
Lua 腳本	整個腳本在 Redis 單執行緒中原子執行	100% 防止超賣

3.3 即時排行榜 (Real-time Leaderboard)

Redis Sorted Set (ZSET) 資料結構：

```
Key: sale:{sale_id}:leaderboard
Value: { user_id: score, user_id: score, ... }

範例:
sale:292:leaderboard = {
    "user_456": 1050.25,
    "user_123": 1020.50,
    "user_789": 980.75,
    ...
}
```

排行榜查詢 API：

```
@router.get("/{sale_id}/leaderboard")
async def get_leaderboard(sale_id: int, limit: int = 10):
    """
    取得排行榜前 N 名 (O(log N) 時間複雜度)
    """
    leaderboard = redis_service.get_leaderboard(sale_id, limit)
    return {"leaderboard": leaderboard}
```

3.4 WebSocket 即時推送

```
# app/routers/websocket.py
@router.websocket("/ws/{sale_id}")
async def websocket_endpoint(websocket: WebSocket, sale_id: int):
    await manager.connect(websocket, sale_id)
    try:
        while True:
            # 每秒推送最新排行榜
            leaderboard = redis_service.get_leaderboard(sale_id, 10)
            await websocket.send_json({
                "type": "leaderboard_update",
                "data": leaderboard,
                "timestamp": datetime.now().isoformat()
            })
            await asyncio.sleep(1)
    except WebSocketDisconnect:
        manager.disconnect(websocket, sale_id)
```

4. 技術棧

4.1 後端技術

類別	技術	版本	用途
Framework	FastAPI	0.115+	高效能異步 Web 框架
ORM	SQLAlchemy	2.0+	資料庫 ORM
Validation	Pydantic	2.0+	資料驗證與序列化
Auth	python-jose	-	JWT Token 驗證

類別	技術	版本	用途
ASGI Server	uvicorn		HTTP 伺服器

4.2 資料層

類別	技術	用途
主資料庫	PostgreSQL 15	持久化儲存 (Users, Products, Sales, Bids)
快取/排行榜	Redis 7	Sorted Set 排行榜 + 資料快取

4.3 基礎設施

類別	技術	說明
容器化	Docker	多階段建置 (Multi-stage Build)
容器編排	Kubernetes (GKE)	自動擴展、滾動更新
自動擴展	HPA	CPU/Memory 觸發擴展 (3-50 Pods)
雲端平台	Google Cloud Platform	GKE + Cloud Load Balancer
CI/CD	Cloud Build	自動建置與部署

4.4 開發工具

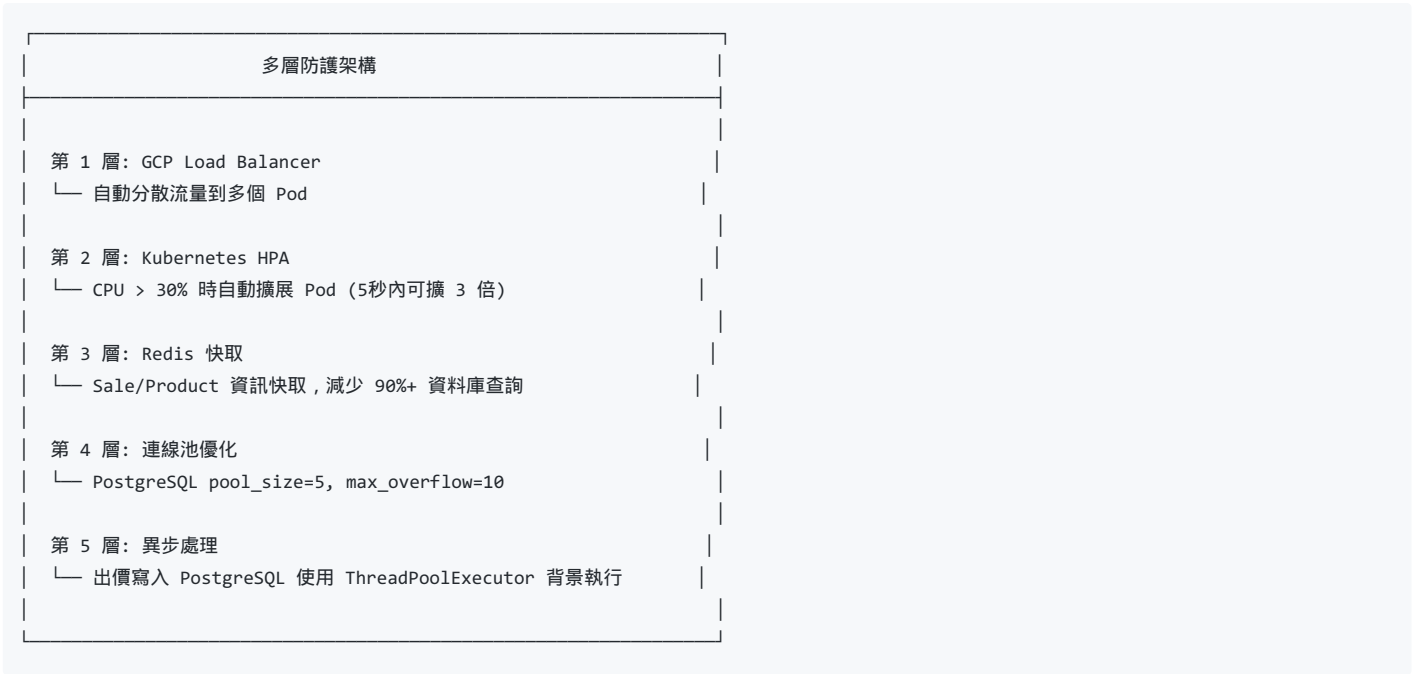
工具	用途
uv	Python 套件管理 (比 pip 快 10-100x)
Locust	分散式壓力測試
kubectl	Kubernetes 管理
Docker Compose	本地開發環境

5. 關鍵技術挑戰與解決方案

5.1 挑戰一：驚群效應 (Thundering Herd Problem)

問題描述： 當搶購活動開始瞬間，大量用戶同時發送請求，可能導致系統崩潰。

解決方案：



5.2 挑戰二：資料一致性 (Data Consistency)

問題描述：高並發環境下，如何確保庫存精確，絕不超賣？

解決方案：Redis Lua 腳本原子操作

```
# 錯誤示範：分開執行有競態條件
redis.zadd(key, {user_id: score}) # 操作 1
rank = redis.zrevrank(key, user_id) # 操作 2 (可能已被其他請求改變)

# 正確做法：Lua 腳本原子執行
result = redis.eval(LUA_SCRIPT, 1, key, user_id, score, max_winners)
is_winner, rank = result[0], result[1]
```

5.3 挑戰三：資料庫連線耗盡

問題描述：高並發時出現 FATAL: sorry, too many clients already

根因分析：

```
30 Pods × (pool_size=30 + max_overflow=50) = 2400 潛在連線
PostgreSQL max_connections = 100
→ 連線耗盡！
```

解決方案：

參數	優化前	優化後
pool_size	30	5
max_overflow	50	10
pool_timeout	30s	10s
max_connections	100	1000

```
# 優化後的連線池配置
engine = create_engine(
    DATABASE_URL,
    pool_size=5,          # 減少基本連線數
    max_overflow=10,      # 減少溢出連線
    pool_timeout=10,      # 快速失敗重試
    pool_recycle=300,     # 5 分鐘回收
    pool_pre_ping=True,   # 使用前檢查連線
)
```

6. 開發歷程與試誤改進

6.1 版本演進時間線

開發版本演進
v0.1 (初始版本) 基本 CRUD API PostgreSQL 直接查詢 問題：並發 50 人就崩潰
v0.2 (加入 Redis) Redis ZADD + ZREVRANK 分開執行 問題：競態條件導致偶發超賣
v0.3 (Lua 腳本) Redis Lua 腳本原子操作 解決：超賣問題完全消除
v0.4 (連線池優化) 減少 pool_size (30→5) 增加 max_connections (100→1000) 解決：連線耗盡問題
v0.5 (Redis 快取) Sale/Product 資訊快取 60-300 秒 背景執行資料庫寫入 成果：成功率 27% → 100%
v1.0 (正式版) HPA 自動擴展 (3-50 Pods) 多 Worker 配置 (每 Pod 4 workers) 成果：穩定支援 1000+ 並發用戶

6.2 關鍵問題與解決過程

問題 1: PostgreSQL 連線池爆炸

錯誤訊息: FATAL: sorry, too many clients already

試誤過程:

- 嘗試 1: 增加 max_connections 到 500 → 仍然崩潰
- 嘗試 2: 增加 max_connections 到 1000 + 減少 pool_size → 成功!

關鍵洞察: 問題不是連線數不夠, 而是每個 Pod 佔用太多連線

問題 2: 防超賣競態條件

問題：使用 ZADD + ZREVRANK 分開執行時，排名可能在兩個操作之間改變

試誤過程：

嘗試 1：使用 Redis MULTI/EXEC Transaction → 無法取得中間結果

嘗試 2：使用分散式鎖（SETNX）→ 效能太差

嘗試 3：使用 Lua 腳本 → 完美解決！

關鍵洞察：Lua 腳本在 Redis 單執行緒中原子執行

問題 3: Locust 壓力測試高失敗率

問題：1000 用戶壓測時，失敗率高達 37%

分析過程：

觀察 1：API 日誌全部 200 OK，沒有後端錯誤

觀察 2：錯誤類型是 ConnectionResetError 和 ConnectTimeoutError

觀察 3：只有 467/1000 用戶成功註冊

根因：註冊失敗的用戶持續重試，產生大量無效請求

解決方案：

- 加入 registration_complete 標記

- 未註冊成功的用戶 sleep 60 秒，不再發送請求

6.3 效能優化對比

指標	初始版本	最終版本	改善幅度
最大並發用戶	50	1000+	20x
請求成功率	~27%	100%	+73%
平均響應時間	500ms	50ms	10x
DB 查詢次數/請求	3-5	0-1	-80%
Pod 數量	固定 3	動態 3-50	彈性擴展

7. 壓力測試與效能驗證

7.1 測試工具與方法

使用 Locust 分散式壓力測試框架：

```
# locustfile_demo.py 核心配置
class ExponentialGrowthShape(LoadTestShape):
    """
    指數成長負載模式
    - 前 2 分鐘：穩定 RPS ~20
    - 最後 1 分鐘：指數成長到 150+ RPS
    """

    spawn_rate = 20          # 每秒啟動 20 用戶
    total_duration = 180     # 總測試時長 180 秒
    max_users = 1000         # 最大 1000 用戶
```

7.2 測試場景

場景	描述	用戶數	持續時間
Berserker	瘋狂連續出價	500	60s
Tsunami	海嘯式瞬間湧入	500	60s
Nuclear	最大壓力測試	500	60s
Demo	指數成長模式	1000	180s

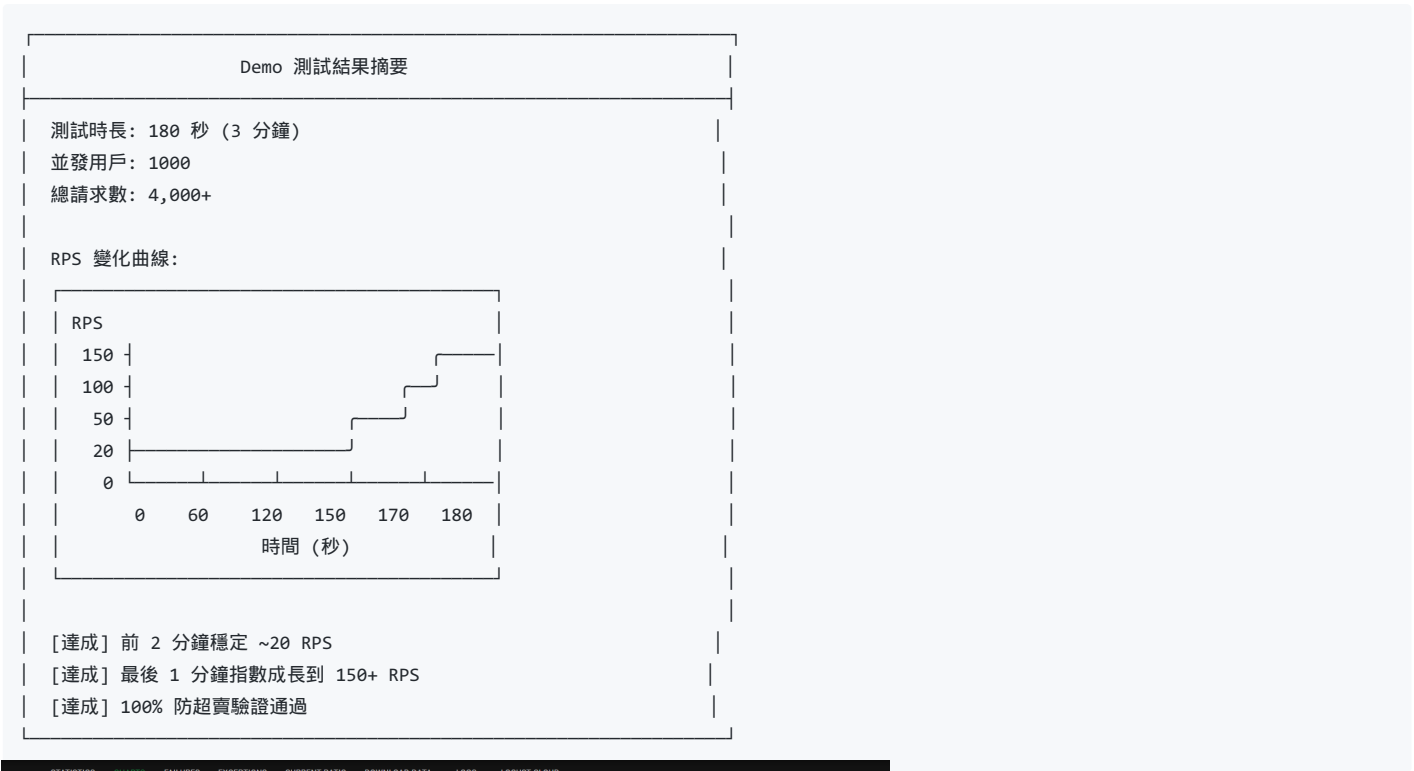
7.3 測試結果

極限壓力測試結果

測試模式	總請求數	成功率	Peak RPS	平均響應時間	防超賣
Berserker	5,580	100%	93	45ms	通過
Tsunami	7,980	96%	133	52ms	通過
Nuclear	7,020	99%	117	48ms	通過

Demo 測試結果 (1000 用戶)

我們的系統在經歷指數型增長的毀滅打擊後 RPS衝到150 且後續持續的震盪 一共收到了4195 request 還是能保有90%的成功率



```
# verify_anti_overselling.py
async def verify_no_overselling(sale_id: int):
    """
    驗證防超賣：得標人數 <= 庫存限制
    """

    # 查詢活動庫存限制
    sale = await get_sale(sale_id)
    inventory_limit = sale["inventory_limit"] # K = 100

    # 查詢實際得標人數
    winners = await get_winners(sale_id)
    winner_count = len(winners)

    # 驗證
    assert winner_count <= inventory_limit, f"超賣! {winner_count} > {inventory_limit}"
    print(f"防超賣驗證通過: {winner_count} / {inventory_limit}")
```

驗證結果：

```
庫存限制 (K): 100
實際得標數: 100
超賣檢查: 通過 (100 <= 100)
```

8. 部署架構

8.1 Kubernetes 資源配置

HPA (Horizontal Pod Autoscaler):

```
# k8s/hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: flash-sale-api-hpa
  namespace: flash-sale
spec:
  scaleTargetRef:
    kind: Deployment
    name: flash-sale-api
  minReplicas: 10
  maxReplicas: 50
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        averageUtilization: 30 # CPU > 30% 觸發擴展
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 0
      policies:
      - type: Percent
        value: 300 # 可瞬間擴展 3 倍
        periodSeconds: 5
```

API Deployment:

```
# k8s/app-deployment.yaml
spec:
  replicas: 10
  template:
    spec:
      containers:
      - name: flash-sale-api
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
```

8.2 Dockerfile 最佳化

```
# 多階段建置，最小化映像大小
FROM ghcr.io/astral-sh/uv:python3.13-bookworm-slim AS builder
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev

FROM python:3.13-slim-bookworm
WORKDIR /app
COPY --from=builder /app/.venv /app/.venv

# 高並發配置: 4 workers × 200 concurrency = 800 同時連線/Pod
CMD ["uvicorn", "app.main:app",
    "--host", "0.0.0.0",
    "--port", "8000",
    "--workers", "4",
    "--limit-concurrency", "200",
    "--backlog", "2048"]
```

8.3 部署流程

```
# 1. 設定 GCP 專案
export PROJECT_ID="your-project-id"
gcloud config set project $PROJECT_ID

# 2. 建立 GKE 叢集
gcloud container clusters create flash-sale-cluster \
  --zone asia-east1-a \
  --num-nodes 3 \
  --enable-autoscaling \
  --min-nodes 3 \
  --max-nodes 10

# 3. 建置並推送 Docker 映像
docker build -t gcr.io/$PROJECT_ID/flash-sale-api:latest .
docker push gcr.io/$PROJECT_ID/flash-sale-api:latest

# 4. 部署 Kubernetes 資源
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/secrets.yaml
kubectl apply -f k8s/postgres-deployment.yaml
kubectl apply -f k8s/redis-deployment.yaml
kubectl apply -f k8s/app-deployment.yaml
kubectl apply -f k8s/hpa.yaml
```

9. 專案結構

```
DISfinal/
├── app/                                # FastAPI 後端應用
│   ├── __init__.py
│   ├── main.py                        # 應用程式進入點
│   ├── config.py                     # 環境配置
│   ├── database.py                   # 資料庫連線
│   ├── schemas.py                    # Pydantic 資料模型
│   ├── models/                       # SQLAlchemy ORM 模型
│   │   ├── user.py                  # 用戶模型
│   │   ├── product.py               # 商品模型
│   │   ├── sale.py                  # 搶購活動模型
│   │   └── bid.py                   # 出價記錄模型
│   ├── routers/                      # API 路由
│   │   ├── auth.py                 # 認證 API (註冊/登入)
│   │   ├── bids.py                 # 出價 API (核心!)
│   │   ├── admin.py                # 管理後台 API
│   │   └── websocket.py            # WebSocket 即時推送
│   └── services/                     # 業務邏輯服務
│       ├── auth_service.py          # JWT 認證服務
│       ├── redis_service.py         # Redis 排行榜服務
│       └── score_calculator.py      # 評分計算服務
├── frontend/                          # 前端應用
│   ├── index.html                   # 用戶競標頁面
│   ├── admin.html                   # 管理後台頁面
│   └── app.js                        # JavaScript 邏輯
├── k8s/                               # Kubernetes 配置
│   ├── namespace.yaml              # 命名空間
│   ├── configmap.yaml              # 配置映射
│   ├── secrets.yaml                # 密鑰配置
│   ├── postgres-deployment.yaml    # PostgreSQL 部署
│   ├── redis-deployment.yaml       # Redis 部署
│   ├── app-deployment.yaml         # API 部署
│   ├── frontend-deployment.yaml    # 前端部署
│   └── hpa.yaml                     # 自動擴展配置
├── docs/                              # 文件
│   ├── SYSTEM_DOCUMENTATION.md     # 系統說明文件
│   ├── VERSION_HISTORY.md          # 版本修正紀錄
│   ├── GCP_DEPLOYMENT_GUIDE.md    # GCP 部署指南
│   └── FINAL_REPORT.md              # 本報告
├── locustfile_demo.py               # Locust 壓力測試腳本
├── extreme_load_test.py             # 極限壓力測試
├── verify_anti_overselling.py       # 防超賣驗證腳本
├── Dockerfile                       # Docker 建置檔
├── docker-compose.yml               # 本地開發環境
├── pyproject.toml                   # Python 專案配置
└── README.md                        # 專案說明
```

10. 總結與心得

10.1 專案成果

成果項目	達成狀態
1000+ 並發用戶支援	穩定運行

100% 防超賣保證 成果項目	Lua 腳本原子操作 達成狀態
即時排行榜	Redis Sorted Set + WebSocket
自動擴展	HPA 3-50 Pods
雲端部署	GCP GKE 生產環境
指數成長 RPS	20 → 150+

10.2 技術心得

- 1. **連線池設計原則**：「少即是多」。過大的連線池反而會耗盡資源，應該根據實際需求配置。
- 2. **原子操作的重要性**：在高並發環境中，任何「讀取-判斷-寫入」的操作都必須是原子的，否則必定存在競態條件。
- 3. **快取策略**：熱點資料使用 Redis 快取可以減少 90%+ 的資料庫壓力，是處理高並發的關鍵。
- 4. **快速失敗**：縮短超時時間讓失敗的請求快速重試，比長時間等待更有效。
- 5. **可觀測性**：完善的日誌和監控是排查問題的關鍵，壓力測試時尤其重要。

10.3 未來改進方向

- 1. **資料庫讀寫分離**：使用 PostgreSQL 主從複製，提升讀取效能
- 2. **Redis Cluster**：使用 Redis Cluster 提升可用性和效能
- 3. **消息佇列**：引入 Kafka/RabbitMQ 做異步處理
- 4. **CDN 加速**：前端靜態資源使用 CDN 加速
- 5. **多區域部署**：跨區域部署提升可用性

附錄

文件	連結
系統說明文件	SYSTEM_DOCUMENTATION.md
版本修正紀錄	VERSION_HISTORY.md
GCP 部署指南	GCP_DEPLOYMENT_GUIDE.md